

data : processed information (has meaning)
 (4 facts & figures)
 → most valuable assets of any org.

→ Every day billions of records are generated through

Healthcare Systems, online transactions, weather forecasting, mobile apps, e-commerce websites, farming, IoT devices, banking sector, etc.

→ Efficient storage, manipulation of data and processing of that data require advance database systems.

→ The evolution of databases reflects the increasing demand of handling massive volumes of structured, semi-structured and unstructured data with high availability and scalability.

Basics File & System

→ Characteristics

- ① Separate file for each application
- ② Sequential data access
- ③ Minimal security
- ④ Difficulty in data sharing
- ⑤ High maintenance cost

eg. Text file, Excel file (Student.txt, Course.txt, marks.txt)

→ Problems with file system

- ① Data redundancy (same info multiple files)
- ② Data inconsistency (no proper sequence for file manipulation)
- ③ Difficulty in data access
- ④ Poor security
- ⑤ Lack of backup and recovery

→ Advantages

- ① Simple implementation
- ② No cost
- ③ Suitable for small applications
- ④ Minimal hardware requirement

→ Disadvantages

- ① Data redundancy
- ② Data inconsistency
- ③ Poor Security
- ④ Limited scalability
- ⑤ Difficult to maintain
- ⑥ No concurrency controls (multiple users can't work parallelly)

* Database Management System (Software)

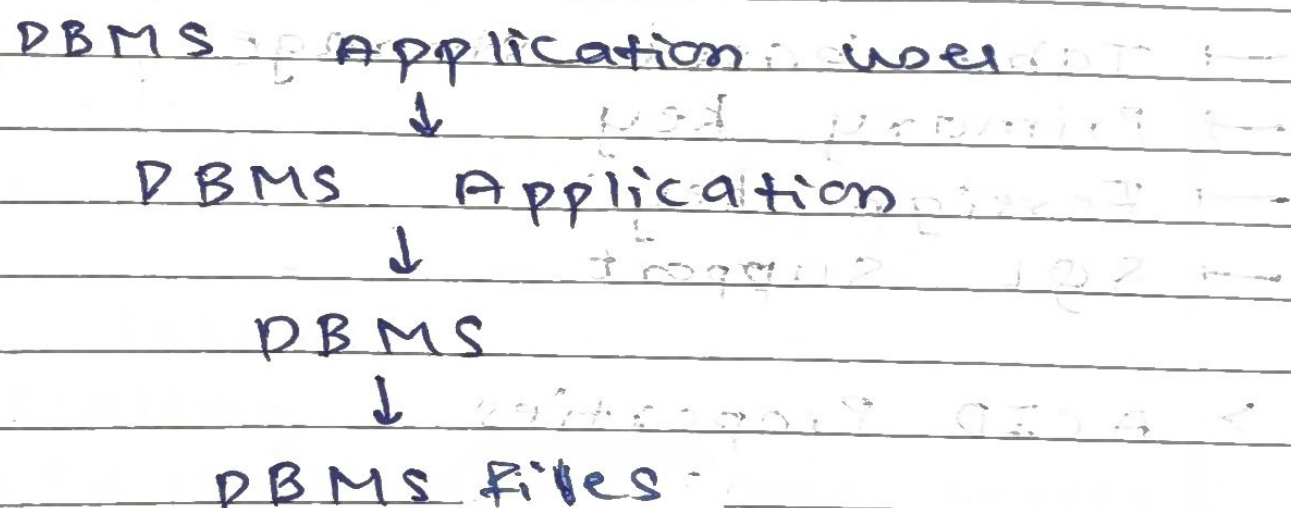
Database Management System is a software that allows users to create, manage and retrieve data efficiently.

eg.

> Characteristics:-

- Reduce Redundancy
- Improve Consistency
- Better Security
- Backup and Recovery
- Concurrent Access and Data Sharing

> Architecture of DBMS



16th July

* Relational Database Management System

Data is stored in the form of tables.

Tables are consisting of rows and columns.

eg.

Roll No.	Student Name	Department
764	Keya	MCA
759	Sneha	BBA
768	Satya	B-Tech

Each row is called a tuple and each column is called an attribute.

> Features

- Table based storage
- Primary key
- Foreign key
- SQL Support

> ACID Properties

- i) Atomicity
- ii) Consistency
- iii) Isolation
- iv) Durability - that guarantees reliable database operations and

maintain data integrity during system crash / power failure.

> Data Integrity

Data integrity in SQL ensures that the data stored in RDBMS remains accurate, consistent, complete and reliable.

> Normalization

→ Normalization in SQL is a systematic database design technique to organize tables in a way that minimizes data redundancy and eliminates undesirable data anomalies during insert, update and delete operations.

eg. MySQL, PostgreSQL, Oracle, Supabase, etc.

> Limitations

→ Scalability Issues

(Vertical Scaling

Adding CPU / RAM on one server)

It's expensive and has practical limits

→ Fixed Schema

(Changing the database schema requires downtime)

→ Performance (bottlenecks)
Handles millions of transactions,
simultaneous users can lead to
slow query performance.

→ Semi-Structured Data.
JSON, Videos, Images

→ High Cost

→ Enterprise RDBMS
Solutions involve significant
licensing and infrastructure cost.

eg. Google, Amazon, & Twitter,
Facebook began generating
petabytes of data daily.

> Characteristics of this data.

→ Massive volume

→ High velocity

→ Wide variety

→ Real time processing needs

→ Global access user

> Big Data

→ Big Data refers to datasets that are too large or complex.

The 5 V's (of big data)

- i) Volume - huge amount of data
- ii) Velocity - high speed of data generation
- iii) Variety - different types of data (Structured, semi-structured & unstructured data)
- iv) Veracity - Reliability and quality of data
- v) Value - Extracting useful insights.

eg. GPS tracking, Social media, etc.

> Why traditional databases are insufficient to handle

- Modern applications require millions of concurrent users.
- 24/7 availability of data
- Global distribution (Horizontal Scalability)
- Fault tolerance
- Flexible Schema

DATE / /

* Questions - Review

1) Explain the limitations of file based systems.

A1) ① Data Redundancy: The same information is often stored in multiple different files, which wastes storage space and can lead to mismatched data if only one file is updated.

② Data Isolation: As data is scattered across various files, it is difficult to retrieve and combine information efficiently.

③ Data Dependence: If the data format is changed, you have to ^{re}write application program.

④ Poor Security: It is very difficult to enforce strict user access controls or protect specific parts of a file.

2) What are the advantages of using a DBMS?

A2) ① Minimal Redundancy: Data is ~~en~~ stored centrally, meaning duplicate entries are controlled and consistency is maintained.

② Data Independence: We can change the database without breaking the software that uses it.

③ Concurrent Access: Multiple users can safely view and update the data at the same time without interfering with each other.

④ Enhanced Security: DBMS provides centralized tools to manage user accounts, passwords and access privileges.

⑤ Backup and Recovery: Data is backed up and restored in case of a hardware or software crash.

3] Differentiate between DBMS and RDBMS?

A3] DBMS

RDBMS

Data Structure > Stores data as files or in hierarchical structures

Relationship > ^{It} Does not support establishing relationships between different files or datasets.

Data Redundancy > Higher chance of data redundancy

Security > Basic Security

Data Volume > Best suited for smaller amounts of data

Feature > Database Management System

Real-world Examples > Windows, XML files, etc.

① > Stores data in a structured, tabular format (Rows & Columns)

> Easily establishes relationships between tables using Primary and Foreign keys

> Minimizes data redundancy

> Advanced security

> Designed to handle massive, complex amounts of data efficiently

> Relational Database Management System

> MySQL, PostgreSQL, Oracle, etc.

4] Discuss the limitations of relational databases in handling Big Data.

A4] ① Scalability Challenges: RDBMS typically scale

"Vertically" (requiring bigger, more expensive servers to handle more data), whereas Big Data requires "horizontal" scaling (distributing load across cheap servers).

② Rigid Schemes: RDBMS requires a fixed, predefined table structure while Big Data is often unstructured or semi-structured.

③ Velocity Issues: Traditional relational databases can struggle to ingest and process the sheer speed and constant flow of real time data generated in Big Data environment.

5) Explain the 5 Vs of Big Data with suitable examples.

A5) ① Volume (Size): The massive scale of data generated globally.
eg. The petabytes of user data stored by Facebook or Google every single day.

② Velocity (Speed): Speed at which new data is created and need to be processed.
eg. Real time credit card fraud detection.

③ Veracity (Fidelity): Accuracy, reliability and trustworthiness of the data, which can often be messy or incomplete.

eg. Filtering out spam, bots or corrupted GPS signals to ensure a dataset is actually useful.

④ Variety (Format): The different forms data takes, including structured, semi-structured, and completely unstructured data.

eg. A company analyzing a mix of neatly formatted sales databases, a raw email text, and audio recording of customer service calls.

⑤ Value (Usefulness): The ability to turn raw data into meaningful, actionable business insights.

eg. Netflix analyzing your viewing habits to recommend a new show.

100

7/10/26

20th
july

* Getting at Persistent Data

- Persistent Data is a data that survives beyond a single run of program or a single session.
- Temporary data exists only while the program is running and when the program ends all the power is cut, the data is usually lost.
- The persistent data is stored in non volatile memory (SSD, HD, Cloud Storage) that keeps information even when power is removed.
- Persistent Data Theory focuses on how digital information maintains its state validity condition and accessibility across time and independent system.
- Programs write ^{persistent} data to storage system such as databases, files or cloud object stores all of which put on non-volatile media.
- These systems are designed so data can be read back later, possibly by different versions of the same application or even by

entirely different applications.
 eg. user accounts in database, log files on drives/disc, ^{backup} cloud & in cloud storage; etc.

→ It only disappears when someone or some process intentionally deletes or overwrites it.

* Why we store data in persistent way?

→ Persistent lets applications remember state overtime such as setting, history, records which is critical for business data like customer information or transactions.

→ It also supports recovery after crashes or power loss because the important data is not tied to in memory variables that vanish when something goes wrong.

→ Without persistent data, every program run would start from scratch with no save progress, references or records.

* Core Architecture

Data : Persistent

Orthogonal persistent

Transient State

* CAP Theorem (used for distributed system theory)

→ It states that a distributed data store can simultaneously provide at most two of the following:

i) Consistency

→ Every read receives the most recent write or an error

ii) Availability

→ The Every non-failing node returns a non-error response without guaranteeing it contains the most recent write operation

iii) Partition Tolerance

→ The system continues to operate despite an arbitrary no. of messages from being dropped or delayed by the network between nodes.

* Concurrency

→ The availability of database to handle multiple data transactions or operations at the exact same time.

* Integration (integration database)

- A database architecture where multiple independent applications directly connect to and share a single monolithic data store.
- It is simple to start with one database everyone uses it.
- Data is all in one place, so you can easily join and query it.

21st * Drawbacks

july

- If one team changes table to support a new feature, another application that relies on the old structure can suddenly stop working.
- Every team becomes dependent on the same database design and release schedule. Small changes require coordination across many groups slowing development.

→ If one big database goes down or becomes very slow, every application that uses it is affected at once.

* Impedance Mismatch

→ Impedance Mismatch is the difficulty of connecting object-oriented code with relational databases.

Object-oriented code

→ Objects can contain another object (nested) or, list ^{and use} inheritance.

In Relational database

→ Data lies in tables ^{in rows} and columns, relationships use foreign keys and join non direct pointers.

* Application Database

→ An alternative pattern to the integration database where a single database is strictly owned by encapsulated within and modified

by one specific application.

* Applications of application database

- Communication between different systems is handled via abstract APIs. Instead of direct database queries, allowing team to change internal database structures cleanly without impacting certain ^{outside} structures.

Q1] Write difference between integration and application database architecture.

DATE / /

* Sharding and Replication in NoSQL cluster.

→ To survive the shift to server cluster, NoSQL database recouple data placement from a single physical machine using two foundation mechanism.

* Sharding

→ Horizontal partitioning of datasets across nodes.

* Hash Routing

→ Deterministic calculation for data placement

• Primary Secondary

→ Single writer topology with multiple readers.

→ Master less design.

→ Multi node concurrent write operation.

→ Synchronization mechanism.

• Quorum Consensus

→ Configurable voting matrices

for read or ^{rand} write consistency.

22nd * NoSQL

July → NoSQL is a category of DBMS designed to store and retrieve large volumes of structured, semi-structured and unstructured data efficiently

* Characteristics

- It is a schemaless or flexible schema.
- Horizontal Scalability
- High availability
- High fault tolerance
- Distributed structure / architecture
- Support for large scale data.
- ~~It~~ Faster read-write operations for specific workload.

23rd july.

* Features of SQL and NoSQL

→ Data Model:

SQL contains schema (tables)

	SQL		NoSQL
Data Model	① Tables	①	Key-Value pairs, (document), Graph, Column
Schema	② Fixed Structure	②	According to requirement flexible

> Scalability ③ Vertical

③ Horizontal

> Transactions ④ ACID properties

④ BASE

> Joins ⑤ Supported

⑤ Limited feature / Avoided

> Query Language ⑥ SQL Language

⑥ As per database specifications

> Performance ⑦ Complex joins

⑦ High Speed read-writes

> Example ⑧ eg. Banking, ERP, etc.

⑧ eg. any Social media, IoT, etc.

* Types of No SQL

① Key-Value Database

→ Data is stored as key-value pair where each key is unique and maps to a single value.

→ The value can be simple datatypes (int, float, string) or complex data structures (list, set).

→ Key value stores high performance and low latency of read or write operations, making them ideal for caching or session.

management and real time analytics where quick access to data is crucial. eg. Riak database, Redis database, Amazon Dynamo DB, etc.

② Document Database

→ Applications:-

- i] Shopping Cart
- ii] Session Management
- iii] Session Storage
- iv] Caching
- v] User Preferences

② Document Database

→ Data is stored as JSON or BSON document format which allows for nested structure and varying fields within each document.

→ Each document is self describing, making it easier to understand and manipulate.

→ The document model supports complex datatypes including arrays and nested objects, allowing for richer data representation and better alignment with object oriented programming paradigm.

→ Document stores provide more advanced query capabilities than key-value stores, allowing for ^{the} searches based on

contents of the documents, how to access content indexing on specific fields and aggregation operations

eg. MongoDB, firebase, CouchDB, etc.

Applications: Amazon, Flipkart, etc.

27th July ③ Column-Family Database

- Data is organized into columns. Families rather than rows.
- In column family stores, data is organized into column families which are collections of related columns that are grouped together.
- Each row within a column family can have a different set of columns allowing for a more flexible schema compared to traditional relational databases.
- Column family stores are optimized for read and write performance, especially for queries that involve large datasets.

eg. Apache Cassandra, HBase, Scylla, etc.

- ④ • Row-oriented / Column Oriented stores are two fundamental approaches to organizing and storing data in a database and each optimize for different use cases.
- Row-oriented stores like traditional relational databases store data in rows meaning all attributes of a single record are stored together. This is particularly efficient for transactional workloads that require quick access to complete records making it ideal for online transactions processing (OLTP) applications.
- Column oriented stores organize data by columns storing all values for a single attribute together. This structure is optimized for analytical queries that often require aggregating or filtering large datasets by specific columns making it suitable for online analytical processing (OLAP) applications.
- As a result, while row-oriented databases excel in read-write operations involving complete records, column-oriented stores provide better performance for complex queries and reporting, particularly when only a subset of columns is needed.

* Aggregate Relationship

- Aggregate Relationship define how different pieces of data within an aggregate relating to one another
- An aggregate is a cluster of related data, can be treated as a single unit encapsulating all necessary information needed to describe a specific entity or a concept.
- eg. user profile, order, product
- In aggregate relationship, an aggregate ~~root~~ (is a) primary entity through which aggregate is accessed or modified.
- It serves as a entry point for operations and ensures that all changes to aggregate maintain its integrity.

(4) Graph Databases

* Applications of column family

- Banking
- Telecommunication
- IoT
- Log Management

28th July → Aggregate is a collection of related objects that we wish to treat as a unit in a particular it is a unit for data manipulation and management of consistency.

→ Typically, we like to update aggregates with atomic operations and communicate with our data storage in terms of aggregates, this definition matches really well with how key-value document and column-family databases work.

→ Dealing in aggregates making it much easier for this database to handle operating on a cluster since the aggregate makes a ^{natural} ~~natural~~ unit for replication and sharding.

* Consequences of Aggregate orientation

→ Consequences of Aggregate orientation for RDBMS

① Relational databases typically require fixed Schema which can lead to complex designs and when trying to model aggregates.

② This can result in numerous tables and relationships making it difficult to manage and evolve the Schema overtime.

③ Aggregates often span multiple tables in RDBMS, necessitating complex joint operations to retrieve related data.

④ This can lead to performance bottlenecks especially when dealing with large datasets or frequent queries.

⑤ The need for multiple queries to fetch related data can degrade performance in relational databases particularly in scenarios where quick access to aggregate data is essential in real time applications.

⑥ Maintaining ACID properties across multiple tables can introduce additional overhead, making transactions more complex and potentially impacting throughput and latency.

→ Consequences for aggregate orientation for NoSQL

① NoSQL databases allow for storing aggregates in a single structure (document) enabling straightforward and efficient retrieval of all related data in one query.

thus improving performance.

- ② The Schema-less nature of many NoSQL databases accommodates dynamic data models, making it easier to adopt to changing requirements without extensive schema alteration.
- ③ NoSQL databases are designed to handle a large volume of data and high level of concurrent request allowing them to scale efficiently while managing aggregates.
- ④ By encapsulating related data within a single aggregate, NoSQL databases minimize or eliminate the need for complex join operations, leading to faster query performance.
- ⑤ Many NoSQL systems adopt an eventual consistency model which can enhance performance and availability but many introduce challenges in maintaining consistency across aggregates in distributed environment.

④ Graph Databases:

- Graph Databases are type of NoSQL database specifically designed to represent and store data in a graph structures which consist of nodes (entities/objects) and edges (relationships)

connecting them

→ They excel in scenarios where relationships are as important as the data itself, such as social networks, recommendation systems, network analysis and fraud detection.

→ One of the key advantages of graph databases is their ability to perform graph complex queries on relationships with high efficiency by leveraging graph.

→ They can quickly retrieve interconnected data without the need of join operations in RDBMS.

Neo4j; Amazon Neptune;

29th

July

→ one of the key ad

→ This capability allows for real-time analytics and insights into data relationships, enabling applications to provide richer context driven user experiences.

→ Popular graph databases: Neo4j, Amazon Neptune, ArangoDB

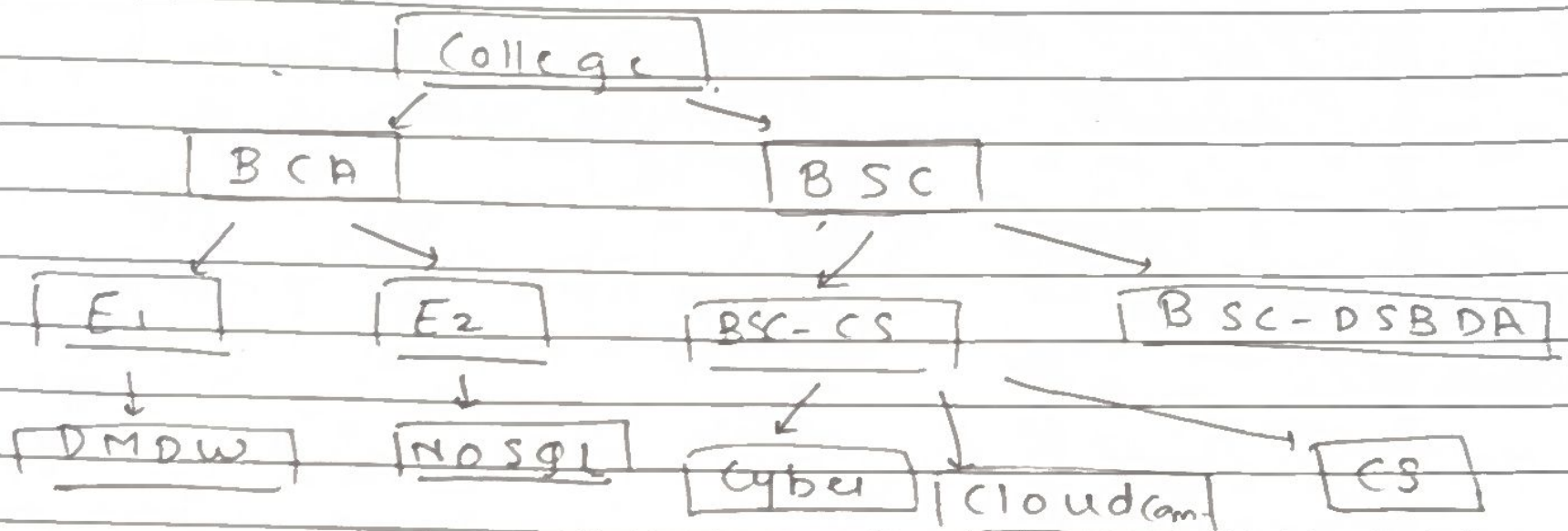
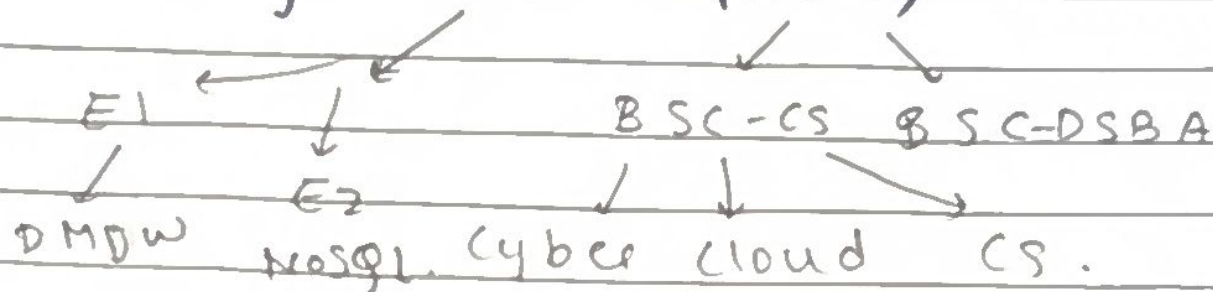
→ As data continues to grow in complexity and interconnectivity, graph databases are increasingly

develop database
according to
query, record

PAGE	found.
DATE	/ /

recognized as essential tools for
managing and analysing relational
data at scale.

eg. College : (BCA, BSc)



* Assignment: 2

- Q1) Define Nosql. Why is Nosql called ~~nosql~~? (not only SQL)?
- Q2) Explain characteristics of Nosql.
- Q3) Differentiate between SQL and Nosql databases.
- Q4) Explain the four types of Nosql databases. 8m

Q5] Explain advantages and limitations of NoSQL databases.

(228, 0.30) : 2.5/10

2

Cassandra

1. It is a distributed database system that stores data in multiple nodes across a network.

2. It is a NoSQL database that uses a column-oriented data model.

3. It is a highly available database that can tolerate multiple node failures.

4. It is a scalable database that can handle large amounts of data.